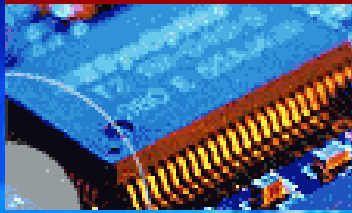
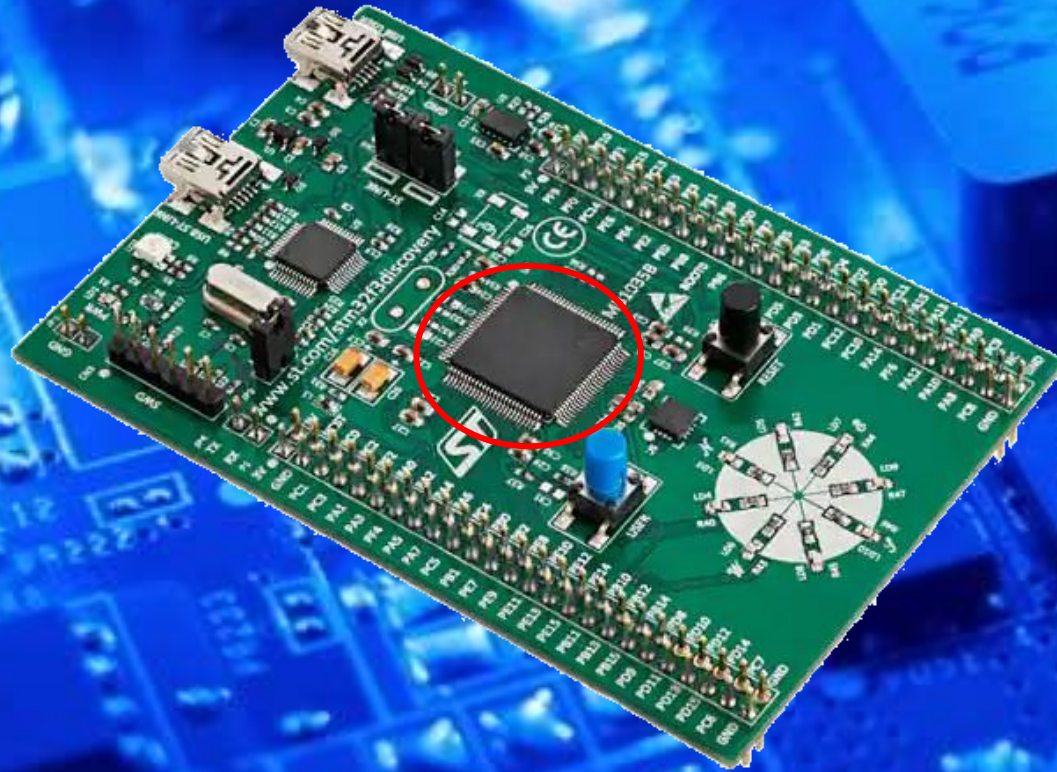


EEE3096S



Embedded Systems II



ARM

Advanced RISC Machines

Part4

ARM Assembly Programming 3/3

Embedded Systems II

L10

Dr Simon Winberg



Electrical Engineering
University of Cape Town

Outline of Lecture

- Reading assignment
- Best Practices for Embedded Software Development
- Mixing C and ARM Assembly
- Implementing an assembly function
- Calling function from C
- Compiling the C and assembly program



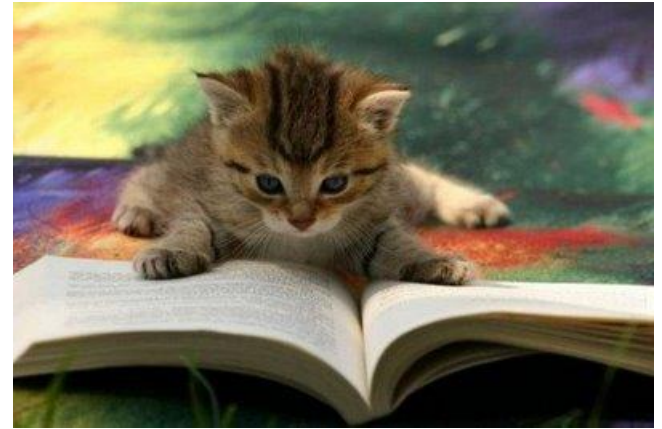
Advanced RISC Machines

The image features a blue border with a white circuit board pattern, including lines, circles, and dots, framing a central white rectangle. Inside the rectangle, the text is centered and reads:

ARM Assembly Language Programming Strategy

Reading Assignment

- Read up on object and executable file formats
- Think about how computers store *programs* on disk and in memory



Read over file **EEE3096S-L10 Reading on Object Files.pdf**
In resources for this course module

Common Assembly Misassumptions

- Avoid making these common assumptions...
 - The hardware definitely works – so I'll write the code, then test the board and code together
 - I can skip commenting this as it's so obvious
 - Changing these two lines won't affect the rest of my program
- Incorrect assumptions can prove costly and time consuming. They're learned through experience. ... and you're sure to come up with some more as you gain experience in this type of coding.

Other Common Misassumptions

- 'Everything is plug and play... sure this too!'
- 'The peripheral will surely work fine when I connect it.'
- 'My board is broken because all my connections are right and my software is perfect!'
- 'My Pi is bad (it's gone vrot) because my code is flawless and yet it still doesn't work 😞'

Strategies for C & Assembly Development

- When programming in C, take small steps
 - Think after every step
 - A bit of Thinking is Cheaper and faster than debugging and rewriting
- When programming in assembly, take tiny steps
 - Assembly language programs are subtle and quick to anger (like wizards)



Mixing C and Assembly



Why Mix C and Assembly

- Interfacing an assembly program with a C library (or vice-versa)
- Performance – writing speed-critical key parts of a program in assembly, and leaving the rest in C, may significantly boost overall performance.
- Size – human written assembly can be smaller than compiler generated code
- *Main reason:* because nowadays, if you ever write any assembly it's usually a tiny piece of your (larger C) program that does something very fast and specialized.

Functions in Assembly

- You can't specify a prototype in the assembler
- You need to ensure that the assembly implementation matches the prototype the C compiler is using
- The interface between a C function and an assembly function can be a good hiding place for bugs – some care is required
- You need to ensure you are using the same Application Binary Interface (ABI) as is being used by the C compiler

What is an ABI?

- Put very generally, an Application Binary Interface (ABI) is an interface between two binary program modules*. Where 'modules' are usually functions (as apposed to code modules that store a collection of functions). However 'module' is used in this case because it might not always be functions that are being connected to, you may, for example, be communicating in some way with a control element (e.g. mutex, I/O or data structure, which is not a function but is a form of application element).
- An ABI might have various types of module, in addition to functions, that one can interact with.
- However, in our case we can assume an ABI is defining just how one function invokes another.

* Further details of this concept at https://en.wikipedia.org/wiki/Application_binary_interface

Why do we need to consider the ABI?

- An ABI would need to be adhered to in, for example, a situation where a library module (e.g. strcpy) is connected to and invoked by another function, e.g. a function in your own user program.
- The **adhering to an ABI** (which may or may not follow an officially standardized) **is usually the responsibility of the compiler**. And also the responsibility of the authors of the operating system and other libraries that you are making use of.
- However, an application programmer may have to deal with the adherence to an ABI explicitly. **Particularly when writing assembly language that you want to connect to from compiled high-level (e.g. C) code**. Or when you may be wanting to mix different programming languages... or mix compiled objects generated by different compilers.

API vs ABI

- API = Application Programming Interface
- API defines the interface between software components at the source code level. Usually a list of functions and data structures available in the application library.
- ABI is lower-level, defining the binary or machine code connection to modules in the application

Developing an Assembly Function

- The following slides will show how to put together an assembly function, using the standard GCC C++ ABI

Functions in Assembly (2)

The assembler has some commands which are used to support interfacing:

.global <name> Indicates that *name* is a global symbol, accessible from other modules

.type <name>, function @ Indicates that the symbol *name* is a function

<name>: Tells assembler to create a symbol with *name* at this position (yes, this is just a label)

Functions in Assembly (3)

- In a C header file (e.g. *asm_module.h*):
`char *strcpy(char * a, char* b); // Prototype`
- In the assembly language listing:
.global strcpy
.type strcpy, function
strcpy:
...implementation of the function...

Calling Conventions

definition Calling Convention*:

The way that parameters, return values, local variables and return addresses are handled is called a calling convention.

- Different platforms and operating systems use different conventions
- The calling convention is part of the Application Binary Interface (ABI) being adhered to
- We will describe the calling convention GCC uses on ARM

* The calling convention is just one of potentially many things defined in an ABI.

Dealing with Parameters

- The first 13 input parameters are passed to the function using registers r0 to r12

```
char *strcpy(char *out, char *in);
```

r0



r1



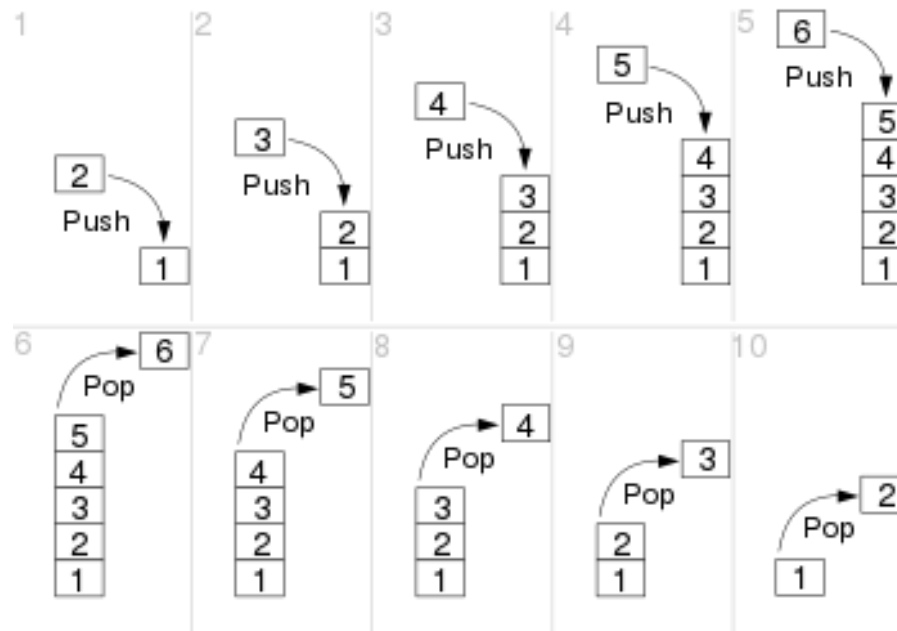
Dealing with Results

- The results are returned in r0

```
char *strcpy(char *out, char *in);
```

r0
↗

Stack Frames



Stack Frames

- The stack space allocated to a specific function is called a stack frame
- The stack frame contains parameters, local variables and the return address
- At the start of the function $FP == SP$
- The stack grows down on the ARM
- SP points to the next free location in the stack



Assembly Function Prologue

i.e. starting the function

strcpy:

@Save sp

mov ip, sp

@stmfd: store multiple, post dec.

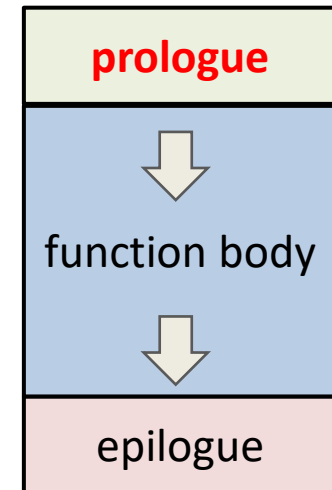
stmfd sp!, {fp, ip, lr, pc}

@fp points to old fp on stack

sub fp, ip, #4

@allocate 4 more words on stack

sub sp, sp, #16



The ARM instruction
 stmfd sp!, {r1, r2}
corresponds to C code:
unsigned int* sp;
unsigned int r1,r2;
*sp = r1; sp++;
*sp = r2; sp++;

Assembly Function Epilogue

i.e. ending the function

strcpy:

... code ...

@pop registers stored on stack

@starting at fp (which we set in prolog)

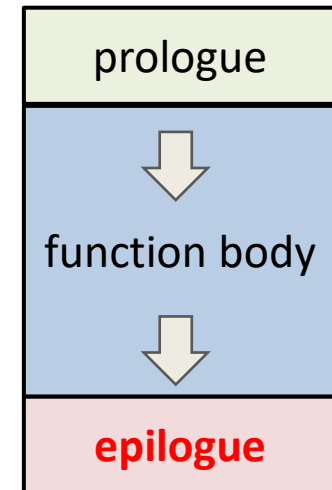
@ldmea = pre-decrement load

ldmea fp, {fp, sp, pc}

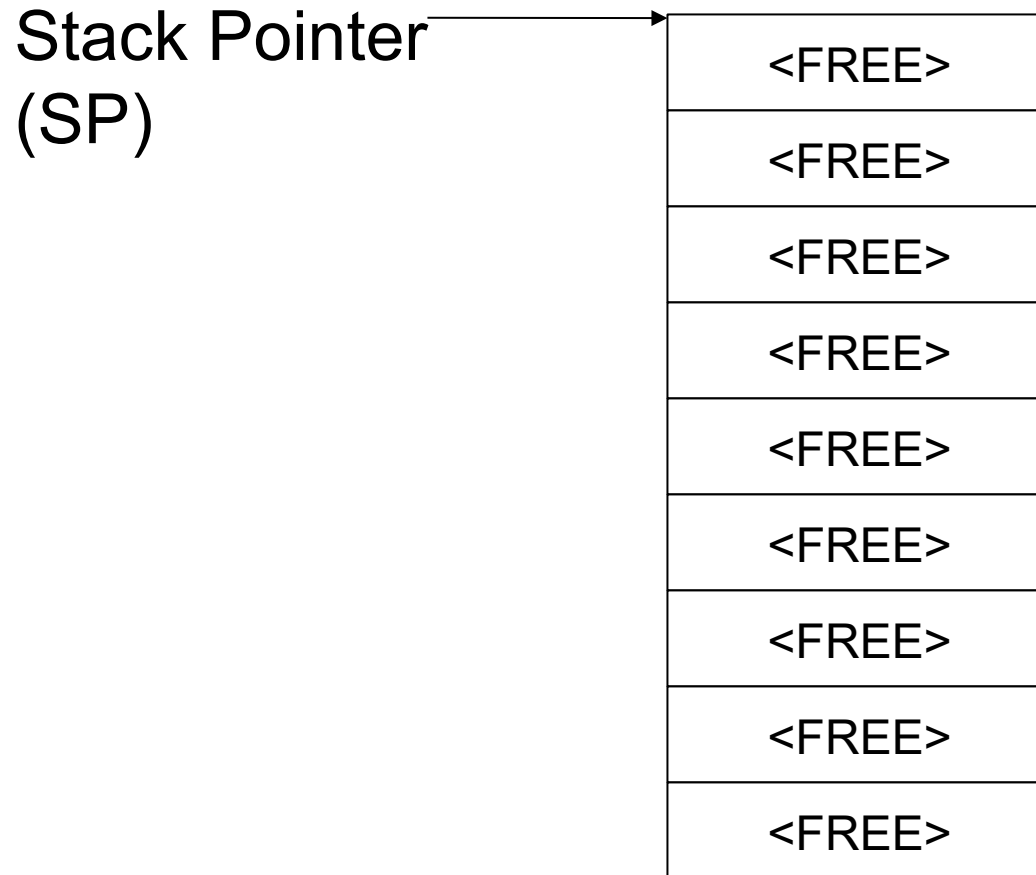
@ note how value of lr gets loaded into pc, undoing the effects of **BL** and returning from the function.

BL = Branch and Link, it is the instruction used to call a function

BL rx does: lr = pc+1, pc=rx, and continues execution from address rx, where rx is a register value. You can branch relative to the pc also (pc=pc+x)



Stack Frame Example



*Instruction: **bl** strcpy*

The above may translate to `bl =strcpy`, loading the 32-bit value from a near-by memory location into the pc register

Stack Frame Example

Stack Pointer
(SP)

<FREE>

<FREE>

<FREE>

<FREE>

<FREE>

<FREE>

<FREE>

<FREE>

<FREE>

Index
Pointer
(IP)

Instruction: **mov** ip, sp

Stack Frame Example

stmfd does:

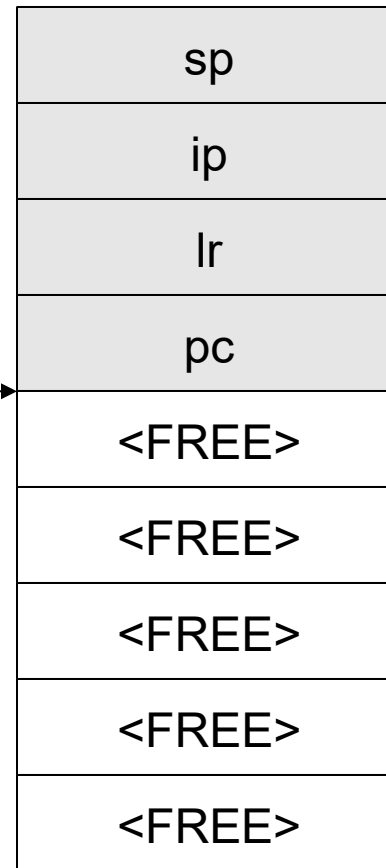
mem[ip]=sp, ip+=4 →

mem[ip]=ip, ip+=4 →

mem[ip]=lr, ip+=4 →

mem[ip]=pc, ip+=4 →

Stack Pointer
(SP) →

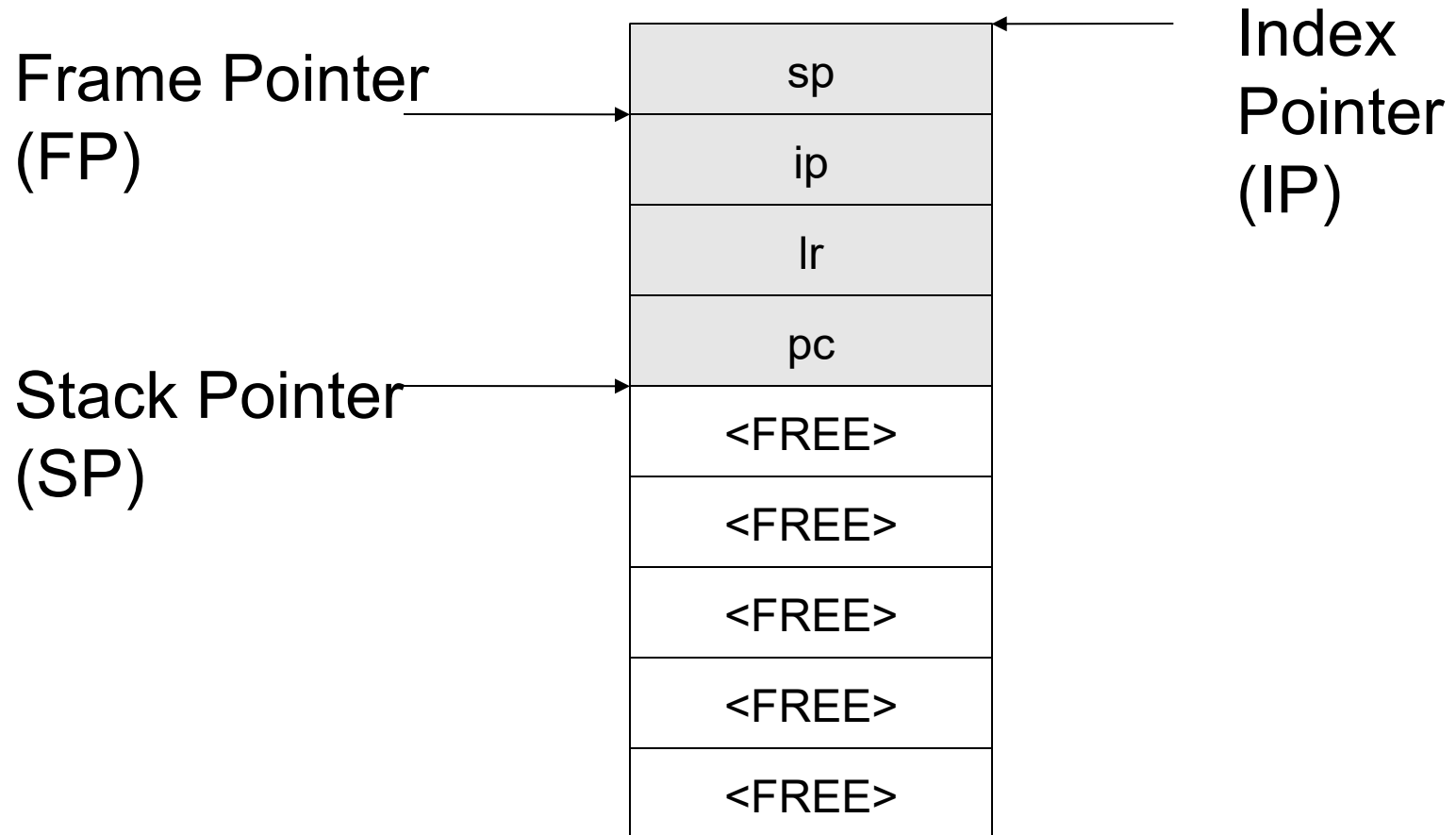


Index
Pointer
(IP) ←

Instruction: **stmfd** sp!, {fp, ip, lr, pc}

i.e. stmfd is the special ARM instruction 'Store Multiple Decrement before Full Descending'. This is essentially a PUSH instruction and in some assemblers, you can use push. This is a special instruction to help with setting the stack frame. ...

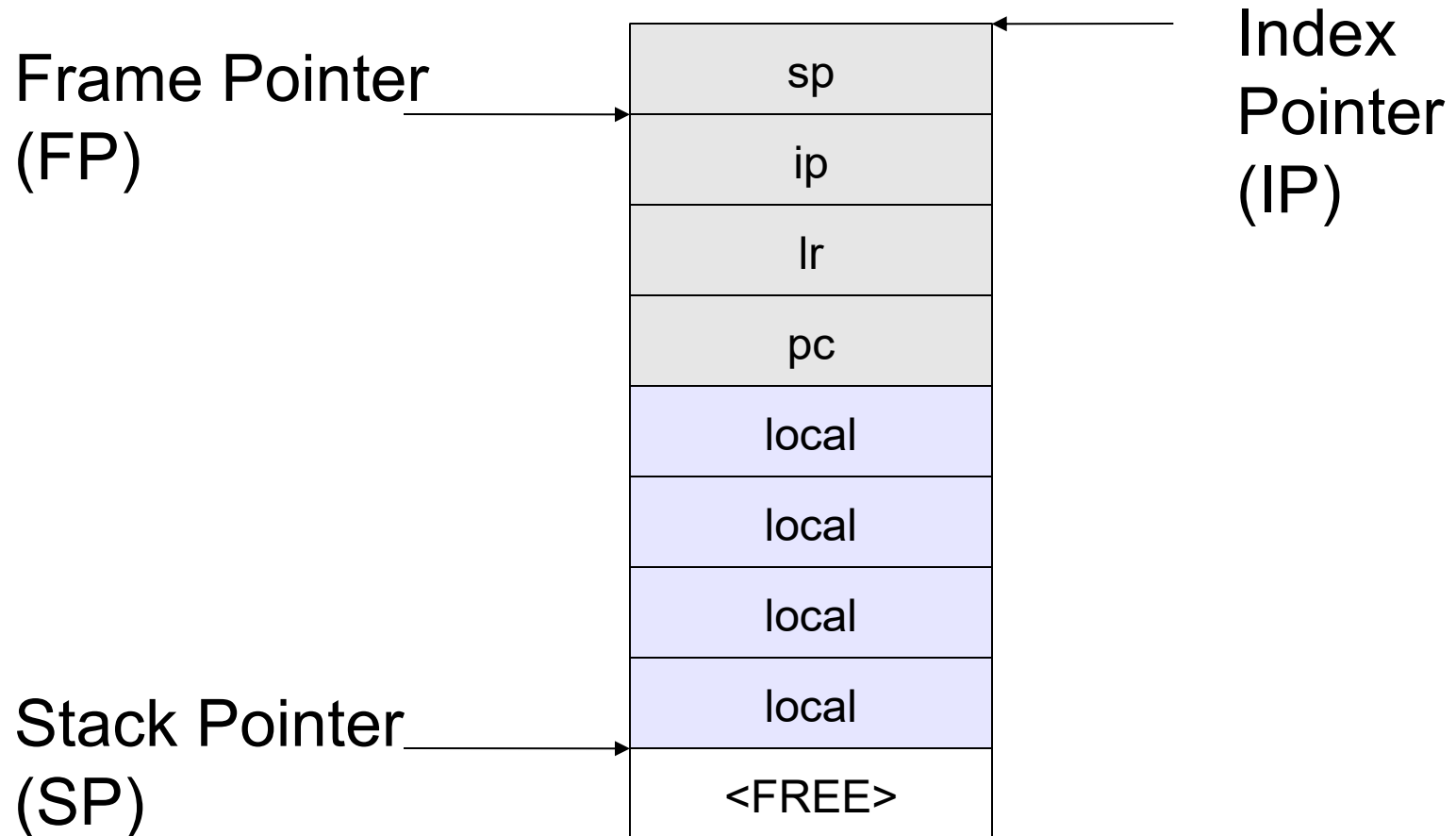
Stack Frame Example



Instruction: **sub** fp, ip, #4

i.e. the new frame pointer is set to the address word after the index pointer

Stack Frame Example



*Instruction: **sub** sp, sp, #16*

i.e. space for local variables are declared by moving sp ahead by the amount needed

Saving Local Variables

There are two basic ways in which you can save local variables so that a given function does not change local variables of the calling function

- Store Before Call (SBC)
(possibly more optimal)

OR

- Call Before Store (CBS) also known as Store Before Use (SBU)
(possibly less optimal but more reliable)

```
void calling_function (void)
{
    int myvar = 100;
    called_function();
    printf("%d",myval); // show 100
}
```

```
void called_function (void)
{
    int v1 = 500;
    int v2 = 300;
}
```

Saving Local Variables

Store Before Call (SBC):

- In this case, the code *before* a function call is design so that all local variables are saved to memory before the function call is made
- This used in some implementations of the -O2 compiler optimization level

Use registers... (e.g., use of r0-r7)

... might have finished with r0,r2,r3,r4

Store registers currently in use (str r1,r5,r6,r7)

Call_function () // uses 20 registers r0-r19

Restore registers that were in use (ldr r1,r5,r6,r7)

FUNCTION INVOKATION OVERHEAD = 4 * 2 store/load = 8 mem accesses

Hint: take note of what the function invocation overhead is and how it is calculated, as this is a useful thing for considering the optimality of your code, and also a favourite type of test question!

Saving Local Variables

Store Before Use / Call Before Store

(typical method used)

- In this case, the code at the start of a function stores all the registers to be used in the function, whether or not the registers were actually used by the calling function

Use registers... (e.g., use of r0-r7)

... might have finished with r0,r2,r3,r4

Only some registers currently in use (i.e. r1,r5,r6,r7)

Call_function () // uses 20 registers r0-r19

store r0-r19

... do body of function

load r0-r19

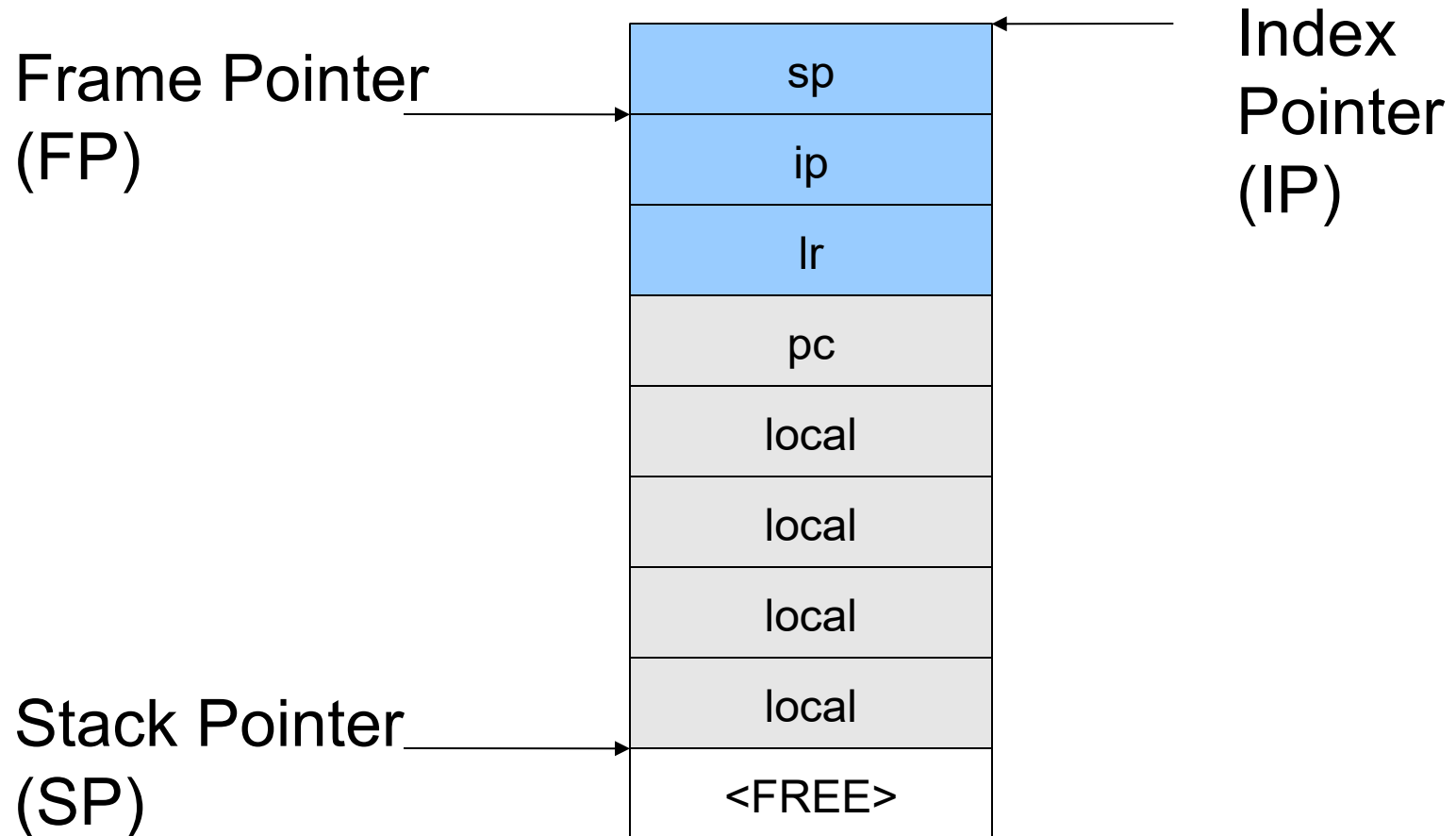
return

Continue using r1,r5,r6,r7 in calling function

i.e., in this example, '*call before use*'
used 32 more memory accesses than
did *store before call*.

FUNCTION INVOKATION OVERHEAD = 20 * 2 store/load = 40 mem accesses

Stack Frame Example



*Instruction: **ldmea** fp, {fp, sp, pc}*

A Real-World Example

Concatenate Strings

- We are going to write a function which adds one string to the end of another
- Similar to one in the C library called `strcat`

A Real World Example

```
char *strcat(char *out, char *in) {  
    int i = 0, j = 0;  
    while (out[i])  
        i++; // Go to end of out  
    while (in[j])  
        // Copy characters from in to out:  
        out[i++] = in[j++];  
    out[i] = 0; // Terminate out string  
    return out; // return the out pointer  
}
```

The Assembly Coding Strategy...

We follow the usual process to implement this as assembly:

1. First you (probably) want to think about the algorithm, what assembly command you are going to use, what parameters, what sort of looping, what registers to use (maybe play around with the design as pen on paper)
2. Once you've decided the registers to use, set up your prologue (you might do the epilogue at the same time)
3. 'Declare' any local variables you may need ... you might not bother to assign them to anything at this point. (I say 'declare' because you're already allocated space for them in step 2, but haven't assigned them values as yet)
4. Implement the routine... obviously this is the hard part, but if you've done #1 effectively it should be pretty mechanical going from pseudocode to instructions.
5. Complete the epilogue and the return operation

Define the C prototype for the assembly

You can tell C about the assembly module by putting a prototype of the function into a h file. We are effectively creating a module. I've called the module strcpyx.h and the assembly file is strcpyx.s (s being the extension for assembly)

```
// File strcpyx.h:  
// Prototype declaration of strcat  
char *strcat(char *out, char *in);
```

Function Prologue

C

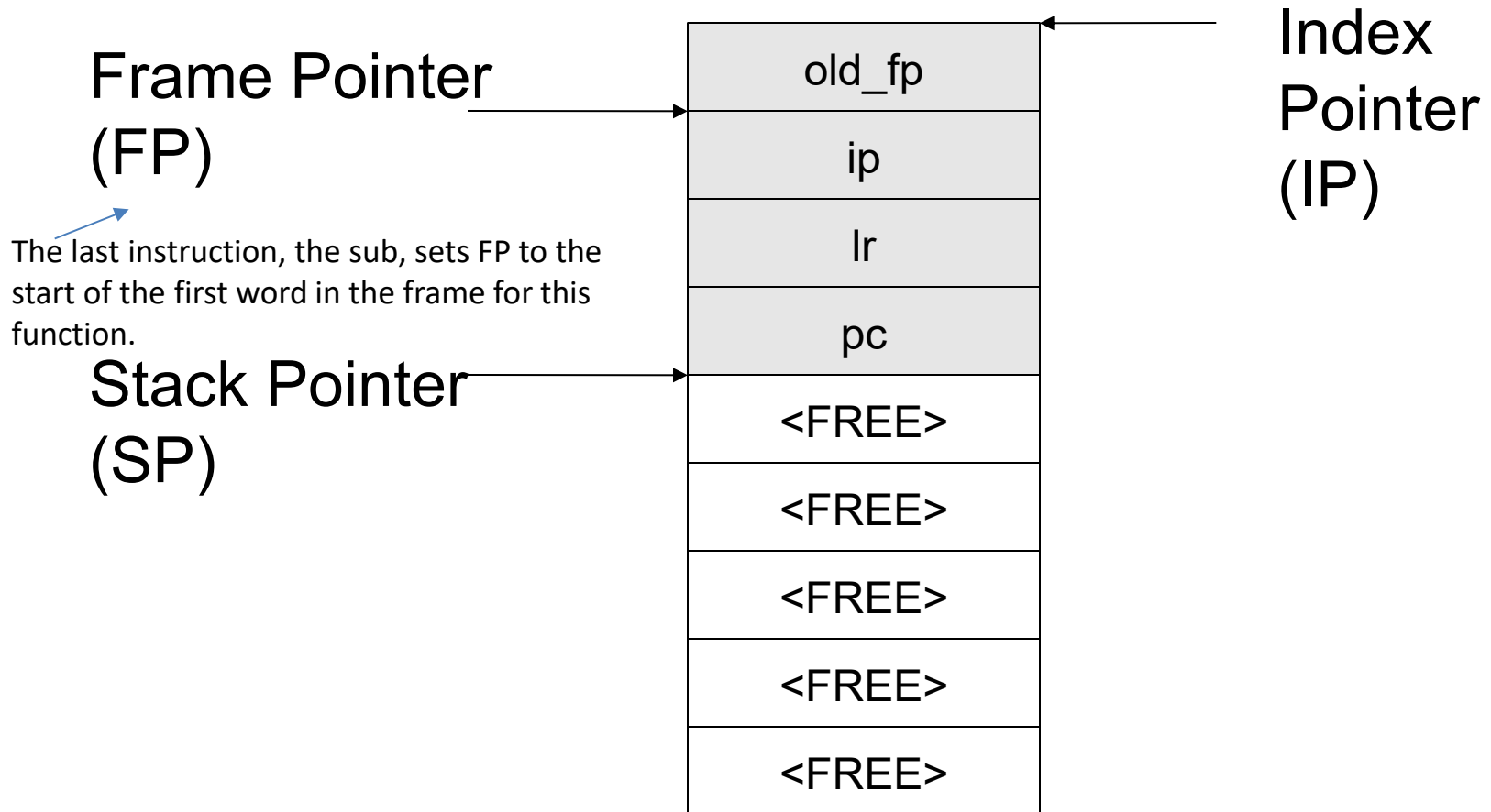
```
char *strcat(char *out, char *in) {
```

ASM

```
.text  
.align 2  
.global strcat  
.type strcat, function  
strcat:  
mov ip, sp  
stmfd sp!, {fp, ip, lr, pc}  
sub fp, ip, #4
```

We are clearly expecting 'store before call' method to be used... the 'store before use' would simply be adding r0-r4 into the register list in the stmfd instruction.

Stack Frame View (at end of prolog)



No space for local variables yet

Declare Local Variables

C

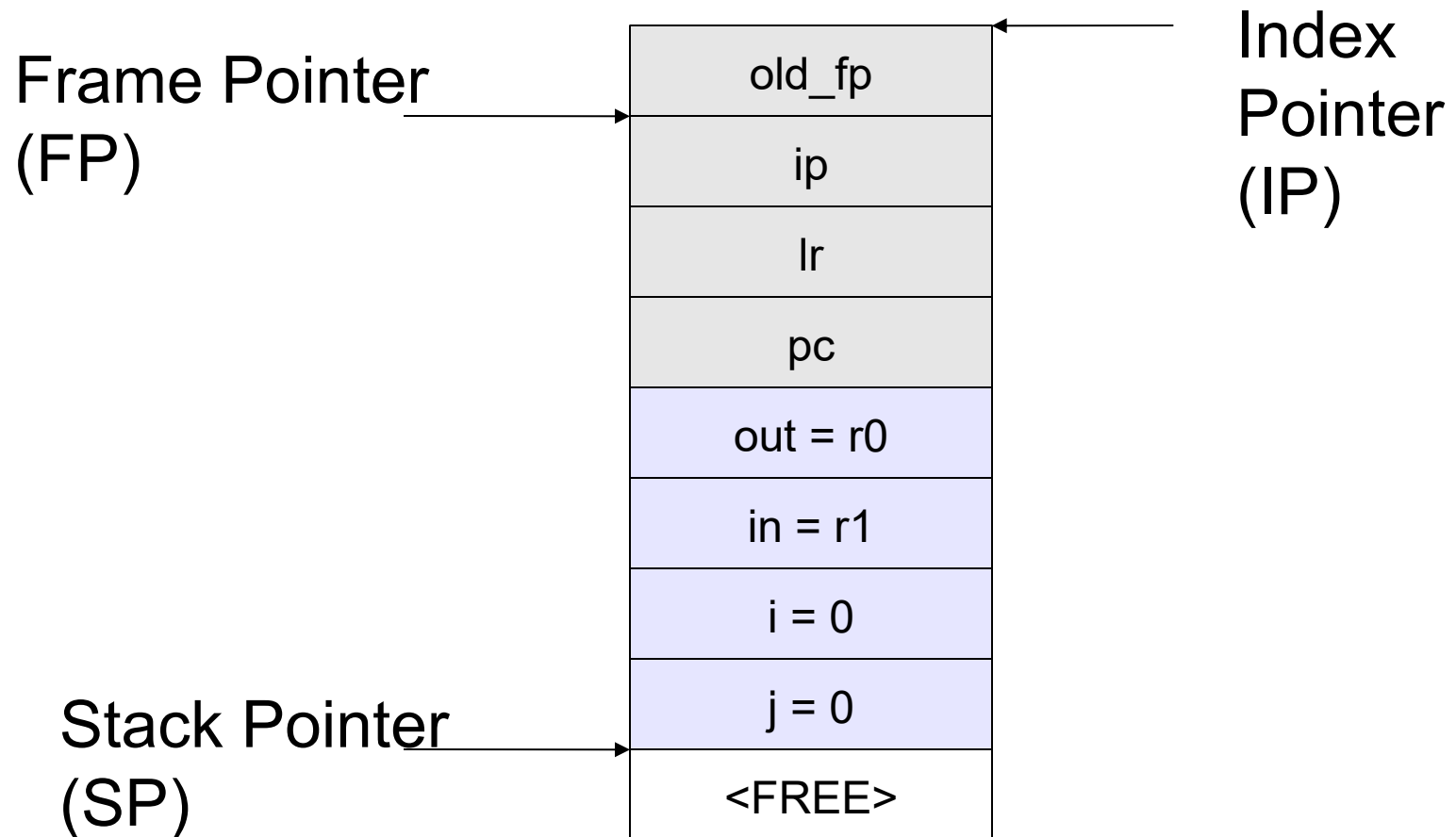
```
int i = 0, j = 0;
```

ASM

```
sub sp, sp, #16 @Space for 4 locals  
str r0, [fp, #-16] @ [fp - 16] = out  
str r1, [fp, #-20] @ [fp - 20] = in  
mov r3, #0  
str r3, [fp, #-24] @ [fp - 24] = i  
mov r4, #0  
str r4, [fp, #-28] @ [fp - 28] = j
```

We directly assign values to the registers (we're assuming 'store before call'), and could copy them through to the stack memory as has been shown (this isn't so optimal unless these registers are needed for other things before getting used in the body of the function.)

Stack Frame View (locals declared)



space and values declared for local variables
values in the registers and also copied to stack memory

Start Processing

C

```
while (out[i]) i++;
```

ASM

```
ldr r2, [fp, #-16]    @ r2 = out
ldr r3, [fp, #-24]    @ r3 = i
loop1:
ldrb r1, [r2, r3]     @ load byte (i.e. out[i])
cmp r1, 0             @ r1 == 0?
be next               @ yes → goto next
add r3, r3, #1        @ no → i++;
b loop1               @ loop back around
next: @end of loop
```

(I applied some optimization / lazy typing by not saving the change to r3, 'i', to address fp, #-24)

Copy String Data

C

```
while (in[j]) out[i++] = in[j++];
```

ASM

Exercise: Write this code.

Tips:

- Load `j` into a register
- Use **ldrb** and **strb** to copy byte data
- Use **cmp** to compare
- Use **be** to branch

Final Part and Epilogue

C

```
out[i] = 0;  
return out;
```

ASM

```
ret_out:  
add r2, r2, r3 @r2 = out + i  
mov r3, #0  
strb r3, [r2, #0] @ out[i] = 0  
ldr r0, [fp, #-16] @ return out pointer  
ldmea fp, {fp, sp, pc} @ do call return
```

Put a '\0' at the end of the concatenated string (the final part of the function body) and the function returns the length of this concatenated string in r0. The last instruction, ldmea, restores the stack frame and returns to the calling function. Notice that again we are assuming the 'store before call' method, we haven't bothered replacing registers used.

Testing the module

- Now you've got the `strcpyx.s` file sorted, and the header file `strcpyx.h`
- Let's connect the `strcpy` in the `.s` file to a C program, i.e. to a `main()` function that will be the entry point of the application.

Test program and compiling it

```
/* EEE3096S Example of combining C and Assembly
 * C entry point for testing the special strcat function */
#include <stdio.h>
#include "strcpyx.h"

void main ()
{
    char a[100] = "Hello "; // set a the a string, make sure there's
    space to add b
    char b[] = "There!"; // set up the b string
    printf("Before:\n"); // print strings before calling function
    printf("a = %s\n",a);
    printf("b = %s\n",b);
    strcat(a,b); // call the strcat we implemented in assembly
    printf("After:"); // print out the result of the strings
    printf("a = %s\n",a); // should get: a = Hello There!
    printf("b = %s\n",b); // b = There!
    printf("Done!\n");
    return;
}
```

Compiling the solution

- First you'd need to make sure you have the right compiler (a GCC-ARM compiler)
- For the Raspberry Pi you can
 - use a cross-compiler (running in Ubuntu to compile for the Pi) as
`arm-linux-gnueabihf-gcc` *
 - Or, running on the Pi directly you can just use the built-in `gcc`
- The command use is explained next slide

Quick instructions for getting this working from pre-build executables are available at:
<https://stackoverflow.com/questions/19162072/how-to-install-the-raspberry-pi-cross-compiler-on-my-linux-host-machine>

Compiling the solution

- To compile on Ubunit / Linux PC use:

```
arm-linux-gnueabihf-gcc -O2 strcpyx.s main.c
```

(note: -O2 optimizations assumes use of SBC calling convention, which we were using)

- To compile directly on the Raspberry Pi use:

```
gcc -O2 strcpyx.s main.c
```

(note assumes O2 optimizations applied to use SBC calling convention)

- Let's see the anticipated results on next page

See example files in `armasm_example1.zip`:

`compile.sh`

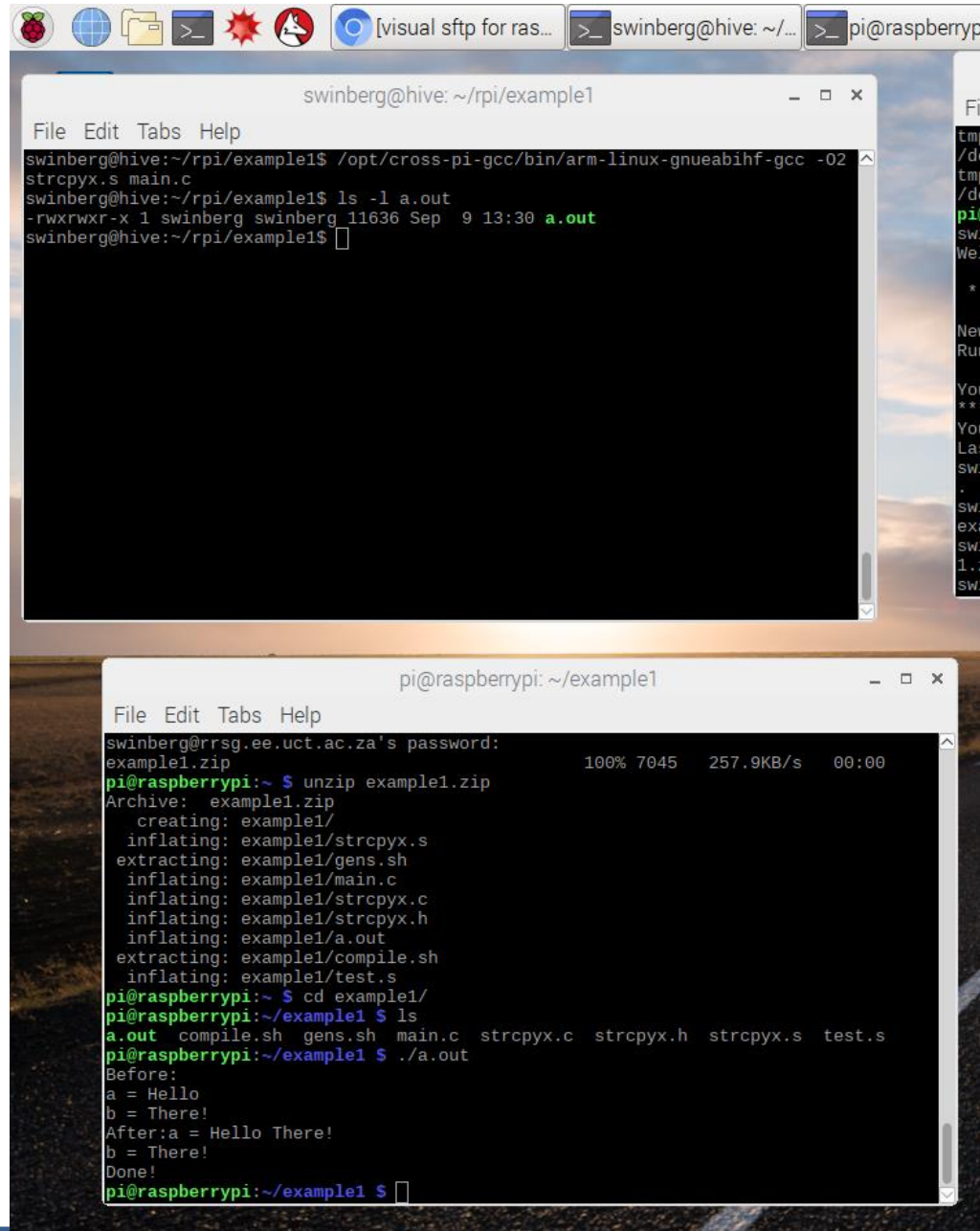
`main.c`

`strcpyx.h`

`strcpyx.s`

`strcpyx.c` (for example sake, not needed)

Example Result



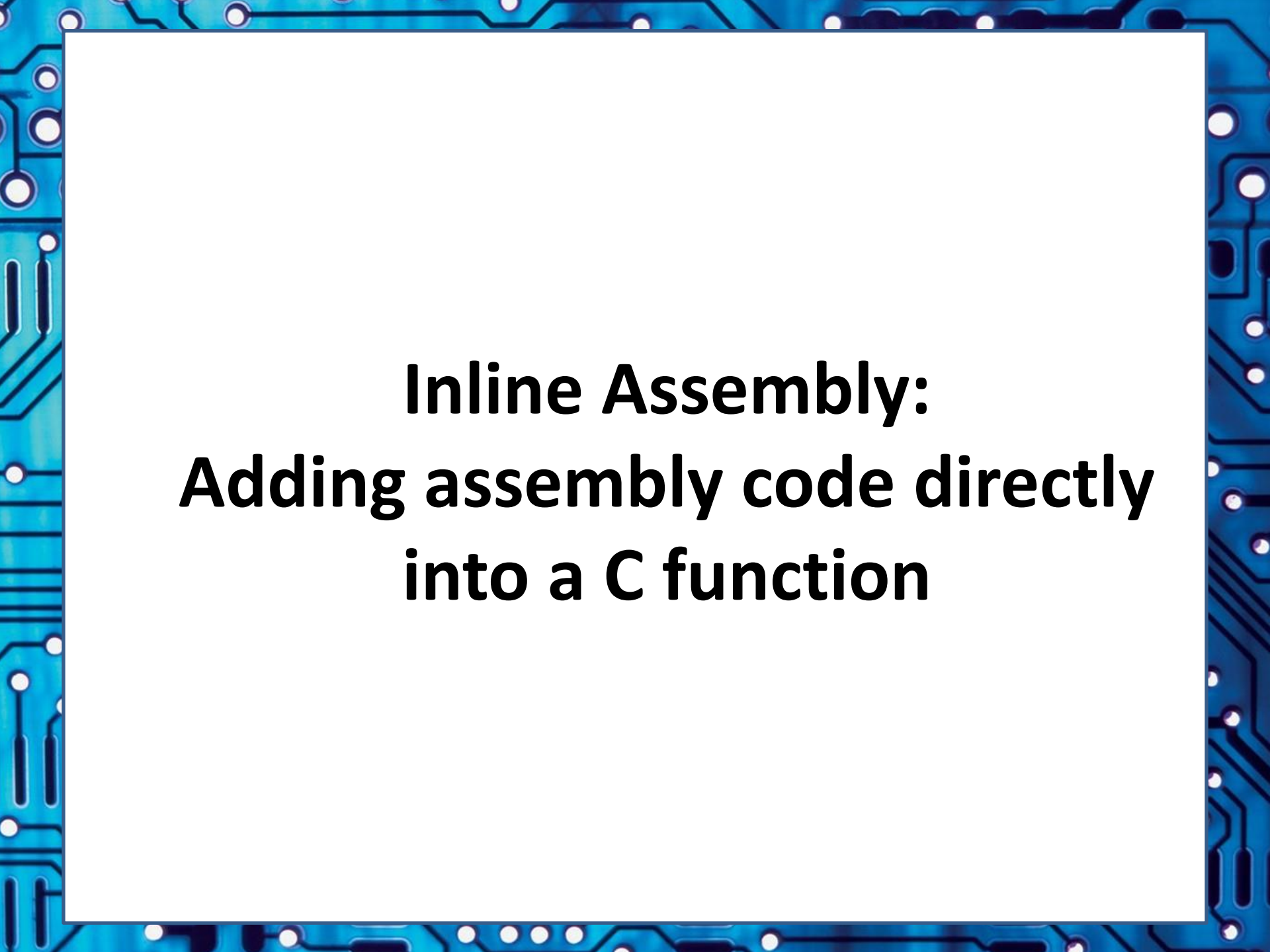
The screenshot shows a terminal window on a Raspberry Pi. The window title is "swinberg@hive: ~/rpi/example1". The terminal output is as follows:

```
swinberg@hive:~/rpi/example1$ /opt/cross-pi-gcc/bin/arm-linux-gnueabi-gcc -O2
strcpyx.s main.c
swinberg@hive:~/rpi/example1$ ls -l a.out
-rwxrwxr-x 1 swinberg swinberg 11636 Sep  9 13:30 a.out
swinberg@hive:~/rpi/example1$
```

Below this, another terminal window is shown with the title "pi@raspberrypi: ~/example1". The output is:

```
swinberg@rrsg.ee.uct.ac.za's password:
example1.zip                               100% 7045  257.9KB/s  00:00
pi@raspberrypi:~$ unzip example1.zip
Archive:  example1.zip
  creating: example1/
  inflating: example1/strcpyx.s
  extracting: example1/gens.sh
  inflating: example1/main.c
  inflating: example1/strcpyx.c
  inflating: example1/strcpyx.h
  inflating: example1/a.out
  extracting: example1/compile.sh
  inflating: example1/test.s
pi@raspberrypi:~$ cd example1/
pi@raspberrypi:~/example1$ ls
a.out  compile.sh  gens.sh  main.c  strcpyx.c  strcpyx.h  strcpyx.s  test.s
pi@raspberrypi:~/example1$ ./a.out
Before:
a = Hello
b = There!
After:a = Hello There!
b = There!
Done!
pi@raspberrypi:~/example1$
```

This screenshot shown that the code was cross-compiled on a PC in Ubuntu, then copied via scp to a Raspberry Pi and run on the Pi, and producing the expected result. Of course you could have just compiled and run the program directly on the Pi.

The background of the slide is a blue circuit board pattern with white lines and dots. A large white rectangle is centered on the slide, containing the text.

Inline Assembly: **Adding assembly code directly** **into a C function**

Inline Assembly

- Assembler code can be included in C code
- Useful for accessing hardware or CPU features that aren't exposed by the C language.
- Inline Assembly is not standardized in ANSI C
- GCC uses the *asm* keyword
 `asm("instruction": inputs: outputs);`
- Other compilers use other keywords and syntax
- Code with inline asm is not ANSI C!

Inline Assembly Best Practice

- Wrap inline assembly in `#ifdef`, `#endif` pair to hide it from other compilers
- Wrap inline assembly in C functions and keep it in a separate file – so the body of your code is still ANSI compliant
- *Some purists say:* Don't use inline assembly at all – rather keep assembly code in a separate module
- Inline assembly can prevent the compiler's optimizer from working – making your code run slower.

GCC Inline Assembly

Example – rotate a value right one bit

```
int rotRight(int val) {  
int result;  
asm("mov %0, %1, ror #1" // i.e. result = val ror #1  
    : "=r" (result)  
    : "r" (val));  
return result;  
}
```

Great document can be found here:

<http://www.ethernut.de/en/documents/arm-inline-asm.html>

Conclusion

Writing assembly code is not particularly hard, but:

- You need a methodical approach
- You need to comment your code – assembly code is much harder to read than C
- Take small steps, and you should be OK!

